

AWS DEVOPS INTERVIEW GUIDE

For every incident, use this interview approach:

Identify → Contain → Investigate → Resolve → Verify → RCA → Preventive Action

Scenario 1: Production Website Suddenly Goes Down (HTTP 503)

Scenario

At 2:00 PM, customers report that the website is unavailable and they receive **503 Service Unavailable** errors.

How would you handle the incident?

Solution

Step 1: Confirm the Issue

Verify:

- Website accessibility
 - Route 53 DNS resolution
 - ALB health
 - CloudWatch alarms
-

Step 2: Check ALB

Verify:

- Target Group Health
- Healthy Hosts
- Health Check Path

If targets are unhealthy:

ALB

↓

No Healthy Targets

↓

503 Error

Step 3: Check Application Servers

`kubectl get pods`

or

`systemctl status application`

Step 4: Check Logs

- CloudWatch Logs
 - Application logs
 - Nginx/Apache logs
-

Step 5: Restore Service

Possible actions:

- Restart application
 - Roll back deployment
 - Scale instances
 - Replace unhealthy instances
-

RCA

Root Cause

Impact

Resolution

Preventive Action

Scenario 2: Entire Kubernetes Cluster is Down

Scenario

All applications running on EKS become unavailable.

Investigation

Verify Cluster

```
kubectl cluster-info
```

Check Nodes

```
kubectl get nodes
```

Possible:

NotReady

Verify AWS

- EC2 instances
 - Auto Scaling Group
 - EKS Control Plane
 - VPC Networking
-

Recovery

- Replace unhealthy nodes
 - Restart node group
 - Restore workloads
-

Prevention

- Multiple AZs
- Managed Node Groups
- Cluster Autoscaler

- Monitoring
-

Scenario 3: Database CPU Reaches 100%

Scenario

Users report slow application response.

Database CPU remains at 100%.

Troubleshooting

Check

CloudWatch

↓

CPU

Connections

Storage

IOPS

Database

Identify

- Long-running queries
 - Locks
 - Deadlocks
 - Missing indexes
-

Resolution

- Kill blocking query
- Add indexes
- Scale RDS

- Enable read replicas
-

Prevention

- Query optimization
 - Performance Insights
 - Monitoring
-

Scenario 4: Terraform Apply Destroyed Production Resources

Scenario

A Terraform deployment accidentally deleted production resources.

Recovery

Immediately

Stop deployment.

Restore

- Terraform State
 - S3 Versioning
 - Backups
-

Redeploy

Infrastructure

↓

Terraform

Prevention

Use

```
lifecycle {  
    prevent_destroy = true  
}
```

Production approval pipeline.

Scenario 5: Jenkins Pipeline Suddenly Starts Failing

Scenario

Every deployment fails.

Investigation

Check

Jenkins

↓

Console Output

Possible causes

- Git authentication
 - Expired AWS credentials
 - Disk full
 - Agent unavailable
 - Plugin update
 - Certificate expiration
-

Resolution

Fix the identified issue.

Restart Jenkins if required.

Prevention

Monitoring

Backups

Pipeline testing

Scenario 6: S3 Bucket Accidentally Deleted

Scenario

Critical production files are deleted.

Recovery

If

Versioning Enabled

↓

Restore Objects

Otherwise

Restore from

- AWS Backup
 - Cross-Region Replication
 - External Backup
-

Prevention

- Versioning
- MFA Delete

- Lifecycle Policies
-

Scenario 7: EKS Pods Enter CrashLoopBackOff

Scenario

After deployment, all pods continuously restart.

Investigation

```
kubectl logs
kubectl describe pod
```

Check

- ConfigMap
 - Secret
 - Database connectivity
 - Memory
 - Image version
-

Resolution

Rollback

```
kubectl rollout undo deployment app
```

Prevention

- Staging testing
 - Health checks
 - Canary deployment
-

Scenario 8: Application Suddenly Becomes Slow

Scenario

Users complain the application takes 30 seconds to respond.

Investigation

Check

- CPU
 - Memory
 - Database
 - Network
 - Load Balancer
 - External APIs
-

Use

- CloudWatch
 - X-Ray
 - Prometheus
 - Grafana
-

Resolution

Scale resources

Optimize queries

Restart unhealthy services

Scenario 9: Production Secrets Exposed

Scenario

AWS credentials were pushed to GitHub.

Immediate Response

Disable

Access Keys

Rotate

- Passwords
 - Secrets
 - Tokens
-

Review

CloudTrail

Prevention

- Secrets Manager
 - GitHub Secret Scanning
 - IAM Roles
-

Scenario 10: Auto Scaling Not Working

Scenario

Traffic increased dramatically, but new EC2 instances were not launched.

Investigation

Check

- CloudWatch Alarm
 - Auto Scaling Policy
 - Launch Template
 - EC2 Limits
-

Resolution

Fix scaling policy.

Increase quotas if needed.

Scenario 11: Route 53 Failover Did Not Work During Regional Outage

Scenario

The primary AWS region went down, but users were not redirected to the DR region.

Investigation

Check:

- Route 53 health checks
 - Failover routing policy
 - TTL values
 - ALB health
 - Secondary region readiness
-

Resolution

Correct the health check configuration, verify DNS records, and ensure the secondary environment is healthy.

Prevention

- Regular DR drills
 - Automated health checks
 - Lower TTL where appropriate
-

Scenario 12: ECS Service Keeps Replacing Tasks

Scenario

An ECS service continuously launches and terminates tasks.

Investigation

Check:

- ECS service events
 - ALB health checks
 - CloudWatch Logs
 - CPU/Memory limits
 - Container exit codes
-

Common Causes

- Application crash
 - Failed health checks
 - Incorrect port mapping
 - Out-of-memory errors
-

Resolution

Fix the application, update the task definition, and redeploy.

Scenario 13: SSL Certificate Expired in Production

Scenario

Users receive browser security warnings because HTTPS certificates have expired.

Investigation

Verify:

- Certificate expiration date
 - ACM status
 - ALB listener configuration
 - DNS validation
-

Resolution

Renew or replace the certificate and update the load balancer.

Prevention

- Use ACM-managed certificates
 - Monitor expiration with CloudWatch/EventBridge notifications
-

Scenario 14: EBS Volume Filled Up

Scenario

An application suddenly stopped writing logs because the EC2 root volume reached 100% utilization.

Investigation

```
df -h
```

Check:

- Log files
 - Temporary files
 - Docker images
 - Container logs
-

Resolution

- Clean unnecessary files
- Extend the EBS volume
- Expand the file system
- Restart affected services if needed

Prevention

- CloudWatch disk alarms
- Log rotation
- Lifecycle management

Scenario 15: High AWS Bill After Weekend

Scenario

Finance reports a significant increase in AWS costs after the weekend.

Investigation

Review:

- AWS Cost Explorer
- Trusted Advisor
- Running EC2 instances
- NAT Gateway usage
- Data transfer
- S3 storage growth

Resolution

Stop unused resources, remove orphaned infrastructure, and optimize scaling policies.

Prevention

- Budgets
 - Cost Anomaly Detection
 - Resource tagging
 - Scheduled shutdown of non-production environments
-

Scenario 16: Docker Image Works Locally but Fails in Production

Scenario

Developers confirm the container works locally, but it crashes after deployment.

Investigation

Check:

- Environment variables
 - Secrets
 - File permissions
 - CPU/Memory limits
 - Network connectivity
 - Base image compatibility
-

Resolution

Update the image, verify runtime configuration, and redeploy.

Prevention

- Test in staging
 - Use identical runtime environments
 - Automated container testing
-

Scenario 17: RDS Storage Almost Full

Scenario

Production database storage reaches 95%.

Investigation

Check:

- Database growth
 - Backup retention
 - Temporary tables
 - Logs
-

Resolution

- Increase allocated storage (or enable storage autoscaling if supported)
 - Archive old data
 - Clean unnecessary records
-

Prevention

- Storage monitoring
 - Data lifecycle policies
 - Capacity planning
-

Scenario 18: Kubernetes Node Becomes NotReady

Scenario

One worker node suddenly becomes unavailable.

Investigation

```
kubectl get nodes
```

Check:

- Kubelet
 - Container runtime
 - EC2 health
 - Disk pressure
 - Memory pressure
-

Resolution

Replace the node or allow the managed node group to recreate it.

Prevention

- Managed Node Groups
 - Cluster Autoscaler/Karpenter
 - Node monitoring
-

Scenario 19: Deployment Introduced a Critical Bug

Scenario

A new release causes users to receive HTTP 500 errors.

Investigation

Compare:

- Previous release
 - Current release
 - Logs
 - Configuration
 - Database migrations
-

Resolution

Immediate rollback:

Kubernetes

```
kubectl rollout undo deployment app
```

ECS

Deploy the previous task definition revision.

EC2

Redeploy the previous application version.

Prevention

- Blue-Green deployment
 - Canary deployment
 - Automated testing
 - Approval gates
-

Scenario 20: Complete AWS Region Failure

Scenario

An entire AWS region becomes unavailable.

Business requires the application to recover within **30 minutes**.

Recovery Plan

Architecture

Users

↓

Route53

↓

Primary Region

↓

Secondary Region

↓

Database Replica

↓

S3 Cross Region Replication

Recovery Steps

1. Route 53 fails over to the secondary region.
 2. Promote the database replica.
 3. Verify application health.
 4. Resume traffic.
 5. Monitor the environment.
-

Prevention

- Multi-region architecture
 - Regular DR testing
 - Infrastructure as Code
 - Automated failover procedures
-

Production Incident Response Framework (Interview Answer Structure)

When asked "**How would you handle this production issue?**", use this structured approach:

Step	Action
1	Acknowledge and assess the impact
2	Identify affected services and severity
3	Contain the issue (prevent further impact)
4	Investigate using logs, metrics, and monitoring tools
5	Restore service as quickly as possible (rollback, restart, scale, failover, etc.)
6	Validate recovery with health checks and user testing
7	Perform Root Cause Analysis (RCA)
8	Implement preventive measures and update documentation

Production Monitoring Tools Commonly Mentioned in Interviews

Tool	Purpose
Amazon CloudWatch	Metrics, logs, alarms
AWS CloudTrail	API audit logs
AWS X-Ray	Distributed tracing

Tool	Purpose
Amazon GuardDuty	Threat detection
Prometheus	Metrics collection
Grafana	Dashboards and visualization
Splunk	Centralized log analysis
Dynatrace	Application performance monitoring (APM)
ELK/OpenSearch	Log aggregation and search
PagerDuty/Opsgenie	Incident alerting and on-call management

High-Impact Interview Tips

When answering production incident questions, emphasize that you would:

- **Prioritize restoring service** before optimizing the system.
- **Communicate regularly** with stakeholders and incident managers.
- **Avoid making risky changes** without understanding the impact.
- **Document timelines and actions** throughout the incident.
- **Conduct a blameless RCA** after recovery.
- **Implement preventive controls** such as monitoring, automation, testing, and runbooks to reduce the chance of recurrence.